Mastering the Tender Management System

Backend Learning Guide & Placement Interview Handbook

Welcome! This handbook is designed for you—a Java developer preparing for placement interviews. It takes you through the **Tender Management System** project, breaking down every technical concept from the ground up.

We won't just look at code; we will understand **WHY** it exists, **WHAT** it does, and **HOW** it works. Let's get started.

Chapter 1: Understanding the Problem

What is a Tender?

Imagine the government wants to build a new highway. The government doesn't have its own construction workers, cement, or bulldozers. Instead, they announce: *"We need a highway built. Who can do it for the best price and quality?"* This official announcement is called a **Tender**.

What is a Vendor?

A **Vendor** is a company or contractor (like L&T or a local construction firm) that provides goods or services. In our project, Vendors are the users who view available Tenders and apply for them.

What is a Bid?

When a Vendor sees a Tender and says, *"I can build this highway for ₹5,000,000 in 6 months,"* that proposal is called a **Bid**. The government (Admin) collects all bids and selects the one with the best balance of cost and timeline.

Why does this software exist? (Real-World Scenario)

Historically, the government procurement process involved massive amounts of paperwork. Vendors had to physically travel to government offices, buy a tender document, fill it out, and drop it in a sealed box. This manual process was slow, prone to corruption, and difficult to manage.

Our **Tender Management System** digitizes this.

- **Admins** can securely float Tenders online.
- **Vendors** can register, browse active Tenders, and submit Bids from anywhere.
- **Admins** can review all Bids and assign the Tender to the best Vendor digitally.

Chapter 2: Complete Project Structure

In Java, we don't just dump all files into one folder. We organize them to keep the code clean, maintainable, and scalable.

1. `src/com/hit/beans`

Why it exists: To hold simple Java objects that carry data across the application. **What's inside:** `VendorBean.java`, `TenderBean.java`, `BidderBean.java`, `NoticeBean.java`, `TenderStatusBean.java`. **Dependency:** Beans don't depend on anything. They are just data carriers.

2. `src/com/hit/dao`

Why it exists: DAO stands for Data Access Object. This folder contains all the code that talks to the MySQL Database. **What's inside:** Interfaces (like `VendorDao.java`) and their implementations (like `VendorDaoImpl.java`). **Dependency:** Depends on Beans (to get/return data) and DBUtil (to get database connections).

3. `src/com/hit/srv`

Why it exists: SRV stands for Servlets. These are the controllers that receive requests from the browser (JSP), process them, and send back a response. **What's inside:** `LoginSrv.java`, `CreateTenderSrv.java`, `BidTenderSrv.java`, etc. **Dependency:** Depends on JSP (receives data from them), Beans, and DAOs (to fetch/save data).

4. `src/com/hit/utility`

Why it exists: To store helper classes that are used everywhere but don't belong to a specific business logic. **What's inside:** `DBUtil.java` (manages database connections) and `IDUtil.java` (auto-generates unique IDs).

5. `WebContent`

Why it exists: To store everything the user sees in their browser. **What's inside:** All `.jsp` files (like `login.jsp`, `adminHome.jsp`), `css`, `images`, `fonts`, and `js`.

6. `WebContent/WEB-INF`

Why it exists: A secure folder that the browser cannot access directly. **What's inside:** `web.xml` (the deployment descriptor that tells the server which file to load first).

Chapter 3: Understanding MVC

MVC stands for **Model-View-Controller**.

Analogy: Think of a Restaurant.

1. **View (Menu/Table):** The customer looks at the menu and places an order. (In our app: **JSP Pages**)
2. **Controller (Waiter):** Takes the order from the customer and gives it to the kitchen. (In our app: **Servlets**)
3. **Model (Kitchen):** Cooks the food using ingredients. (In our app: **Beans and DAOs** interacting with the Database).

How our project uses MVC:

- **View:** A vendor fills out the login form on `login.jsp`.
- **Controller:** The form data is sent to `LoginSrv.java` (Servlet).
- **Model:** `LoginSrv.java` asks `VendorDaoImpl.java` to check the database. `VendorDaoImpl.java` returns a `VendorBean` if the user is valid.

```
graph TD
    A[Browser / Client] -->|HTTP Request| B(Controller: Servlets e.g. LoginSrv)
    B -->|Calls| C(Model: DAO e.g. VendorDaoImpl)
    C <-->|SQL Queries| D[(Database)]
    C -->|Returns Data| E(Model: Beans e.g. VendorBean)
```

```
    E --> B
    B -->|Forwards to| F(View: JSP e.g. vendorHome.jsp)
    F -->|HTTP Response| A
```

Chapter 4: Complete Frontend to Backend Flow

Let's trace the complete lifecycle of a **Login** request.

1. **Browser:** The user opens `login.jsp` and enters their email and password.
2. **JSP:** The form submits a `POST` request to `/LoginSrv`.
3. **Servlet (`LoginSrv.java`):**
    - Acts like a **Receptionist**.
    - Extracts `username` and `password` from the request.
    - Checks if the user is logging in as "Admin" or "Vendor".
4. **DAO (`VendorDaoImpl.java`):**
    - The Servlet calls the Database. Wait, `LoginSrv` actually writes the SQL directly in the servlet for Vendor login (which is a bad practice, but we did it here for simplicity!). It prepares a `SELECT * FROM vendor WHERE vid=? AND password=?`.
5. **DBUtil (`DBUtil.java`):**
    - The Servlet asks `DBUtil.provideConnection()` for an active link to MySQL.
6. **Database:**
    - MySQL executes the query and returns the matching row.
7. **ResultSet:**
    - The result is stored in a `ResultSet` object (like an **Exam Result Sheet**).
8. **Bean (`VendorBean.java`):**
    - The Servlet extracts data from the `ResultSet` and packs it into a `VendorBean` (like packing items into a **Courier Box**).
9. **Session (`HttpSession`):**
    - The Servlet creates a Session (like giving the user a **Hotel Room Key**) and stores the `VendorBean` inside it: `session.setAttribute("vendordata", vendor);`
10. **JSP (`vendorHome.jsp`):**
    - The Servlet forwards the user to `vendorHome.jsp`, which reads the Session and displays "Welcome, [Name]!".

Chapter 5: Servlet Deep Dive

A Servlet is the **Receptionist** of our web application. It takes incoming requests from JSPs, delegates work to the DAOs, and returns the response.

1. `LoginSrv.java`

- **Purpose:** Authenticates Admins and Vendors.
- **Request Parameters:** `username`, `password`, `user` (Admin or Vendor).
- **Business Logic:** Checks if the role is Admin (hardcoded "Admin"/"Admin"). If Vendor, it connects to the DB, checks credentials via `vid` or `vemail`.
- **Session Usage:** Creates `session.setAttribute("user", "user")` and stores the `VendorBean`.
- **Redirects:** Forwards to `adminHome.jsp` or `vendorHome.jsp`. On failure, includes `login.jsp` with an error message.
- **Interview Question:** *Why did you write JDBC code inside LoginSrv instead of VendorDao? Answer:* Ideally, all JDBC code should be in the DAO layer to strictly follow MVC. In this project, `LoginSrv` handles it directly to quickly bind the resulting data into a Session `VendorBean`. However, refactoring this into `VendorDaoImpl.validateLogin()` would be a great future improvement.

2. `RegisterSrv.java`

- **Purpose:** Registers a new Vendor.
- **DAO Calls:** Calls `IDUtil.generateVendorId()` to get a unique ID, creates a `VendorBean`, and passes it to `VendorDaoImpl.registerVendor(vendor)`.
- **Forwarding:** Returns to `login.jsp` with a success/failure message.

3. `CreateTenderSrv.java`

- **Purpose:** Admin creates a new tender.
- **Session Usage:** First checks if `session.getAttribute("user")` is "admin". If not, redirects to `loginFailed.jsp` (Security!).
- **DAO Calls:** Extracts form data, generates Tender ID via `IDUtil`, packs a `TenderBean`, and calls `TenderDaoImpl.createTender()`.

4. `BidTenderSrv.java`

- **Purpose:** Vendor places a bid on a tender.
- **Request Parameters:** `bidamount`, `biddeadline`, `tid`, `vid`.

- **DAO Calls:** Calls `BidderDaoImpl.bidTender()`.

5. `AcceptBidSrv.java` & `RejectBidSrv.java`

- **Purpose:** Admin accepts or rejects a vendor's bid.
- **Business Logic:** `AcceptBidSrv` calls `BidderDaoImpl.acceptBid()`. If accepted, it also calls `TenderDaoImpl.assignTender()` to link the Tender to the Vendor. `RejectBidSrv` simply updates the bid status to "Rejected".

Chapter 6: DAO Layer

DAO (Data Access Object) is the **Bank Cashier**. The Servlet (Receptionist) cannot go into the vault (Database) directly. It must ask the DAO to do it.

1. `VendorDaoImpl.java`

- **Purpose:** Handles all Vendor-related database operations.
- **Methods:** `registerVendor()`, `getAllVendors()`, `validatePassword()`, `updateProfile()`, `changePassword()`.
- **Connection Handling:** Every method calls `DBUtil.provideConnection()` at the start, and uses a `finally` block to call `DBUtil.closeConnection()` to prevent memory leaks.
- **PreparedStatement:** Used everywhere to prevent **SQL Injection** attacks.

2. `TenderDaoImpl.java`

- **Purpose:** Manages Tenders.
- **Key Method:** `assignTender(String tenderId, String vendorId, String bidderId)`. Checks `tenderstatus` table to ensure the tender isn't already assigned before inserting a new record.

3. `BidderDaoImpl.java`

- **Purpose:** Manages the bidding process.
- **Key Method:** `acceptBid()`. It updates the `bidder` table status to 'Accepted' AND calls `assignTender()` to update the `tenderstatus` table.

Why does the DAO exist?

To separate SQL logic from Java Business Logic. If we migrate from MySQL to Oracle tomorrow, we only rewrite the DAO classes; the Servlets remain completely untouched.

Chapter 7: Bean Classes

Beans are **Courier Boxes**. They are simple Java classes with `private` variables and `public` Getters and Setters.

Why Getters and Setters?

To enforce **Encapsulation**. You cannot directly access `vendor.password`. You must use `vendor.getPassword()`. This allows us to add validation or formatting later without breaking other code.

1. `VendorBean.java`

Holds `id`, `name`, `mobile`, `email`, `address`, `company`, `password`. Note that it implements `Serializable`. **Why Serializable?** Because Beans are often stored in HTTP Sessions or sent over networks. Serializable converts the object into a byte stream.

2. `TenderBean.java`

Holds `id`, `name`, `type`, `price`, `desc`, `deadline`, `location`. Notice the constructor: It takes `String stdeadline` and parses it into a `java.util.Date` using `SimpleDateFormat("yyyy-MM-dd")`.

3. `BidderBean.java`

Holds `bidId`, `vendorId`, `tenderId`, `bidAmount`, `bidDeadline`, `bidStatus`. It contains complex Date parsing to convert strings into `java.util.Date` for Java use, and later the DAO converts it to `java.sql.Date` for the database.

How a Bean travels:

Form Data $\rightarrow$ Servlet $\rightarrow$ **Bean** $\rightarrow$ DAO $\rightarrow$ Database. Database $\rightarrow$ DAO $\rightarrow$ **Bean** $\rightarrow$ Servlet $\rightarrow$ Session $\rightarrow$ JSP.

Chapter 8: DBUtil

The `DBUtil` class is our **Electricity Connection**. Without it, the DAOs cannot talk to the Database.

How it works:

1. `ResourceBundle`: Instead of hardcoding the database username and password in Java, `DBUtil` reads them from `src/dbdetails.properties`. This makes it easy to change the password later without recompiling the Java code!
2. **Driver Loading:** `Class.forName(dbDriver)` loads the `com.mysql.jdbc.Driver`. This tells Java we are connecting to a MySQL database specifically.
3. **Connection:** `DriverManager.getConnection()` creates the actual network link to MySQL.
4. **Connection Closing:** It contains overloaded methods to close `Connection`, `PreparedStatement`, and `ResultSet`.

Interview Question:

Why are DBUtil methods static? Answer: By making `provideConnection()` static, we can call `DBUtil.provideConnection()` directly without creating a `new DBUtil()` object every time. It saves memory and makes the code cleaner.

Chapter 9: Database

Our MySQL database is named `tender`. Let's understand the tables.

1. `vendor` Table

- **Purpose:** Stores registered vendor details.
- **Primary Key:** `vid` (e.g., V20190725020951)
- **Used By:** `VendorDaoImpl.java`

2. `tender` Table

- **Purpose:** Stores tenders created by the Admin.
- **Primary Key:** `tid` (e.g., T20190725022124)
- **Used By:** `TenderDaoImpl.java`

3. `bidder` Table

- **Purpose:** Stores bids placed by Vendors on Tenders.
- **Primary Key:** `bid` (Bid ID)
- **Foreign Keys:** `vid` (Links to `vendor`), `tid` (Links to `tender`)
- **Used By:** `BidderDaoImpl.java`

4. `tenderstatus` Table

- **Purpose:** Tracks which Tender is assigned to which Vendor.
- **Primary Key:** `tid` (Foreign Key referencing `tender`)
- **Used By:** `TenderDaoImpl.assignTender()`

5. `notice` Table

- **Purpose:** Stores public announcements by the Admin.
- **Primary Key:** `id` (Auto Increment Integer)
- **Used By:** `NoticeDaoImpl.java`

Chapter 10: Authentication

Authentication is how we verify who the user is.

The Login Flow

1. User selects "Login as Admin" or "Login as Vendor" on `login.jsp`.
2. `LoginSrv.java` checks the database.
3. **Session:** If valid, an `HttpSession` is created.
    - *Analogy:* A Session is a **Hotel Room Key**. HTTP is a stateless protocol (it forgets you the moment a page loads). By putting `session.setAttribute("user", "user")`, the server remembers you across different pages.
4. **Security:** In `vendorHome.jsp` and `adminHome.jsp`, there is a check at the very top:

    ```
    if(!user.equalsIgnoreCase("admin") || uname.equals("") || pword.equals("")){
        response.sendRedirect("loginFailed.jsp");
    }
    ```

    This prevents unauthorized users from accessing the page directly via URL.

5. **Logout:** `LogoutSrv.java` clears the session by setting the attributes to empty strings `""` and redirecting to `logoutSuccess.jsp`. (Ideally, `session.invalidate()` should be used to completely destroy the session).

Chapter 11: Tender Flow

The lifecycle of a Tender in our project:

1. **Create:** Admin fills `createTender.jsp`. `CreateTenderSrv` uses `IDUtil` to generate an ID like `T20190725022124` (T + Current Date/Time). The Tender is stored in the `tender` table.
2. **Display:** Vendors view active tenders via `vendorViewTender.jsp` which calls `TenderDaoImpl.getAllTenders()`.
3. **Bid:** Vendors bid on it.
4. **Assign:** Admin goes to `viewTenderBids.jsp` and clicks "Accept" on the best bid. `AcceptBidSrv` calls `TenderDaoImpl.assignTender()`.
5. **Status Changes:** The `tenderstatus` table gets a new row linking `tid` and `vid` with status 'Assigned'.

Chapter 12: Vendor Flow

1. **Registration:** `register.jsp` $\rightarrow$ `RegisterSrv` $\rightarrow$ `VendorDaoImpl.registerVendor()`. The Vendor ID is auto-generated by `IDUtil`.
2. **Login:** Explained in Chapter 10.
3. **Update Profile:** `updateProfile.jsp` pre-fills data from `session.getAttribute("vendordata")`. Form goes to `UpdateProfileSrv`, updates the DB, and crucially, updates the `VendorBean` inside the current Session!
4. **Change Password:** `ChangePasswordSrv` verifies the old password against the Database before updating it to the new one.

Chapter 13: Bid Flow

1. **How Vendor Bids:** In `bidTender.jsp`, the vendor clicks "Bid". `BidTenderSrv` accepts `bidamount` and `deadline`. `IDUtil.generateBidderId()` generates an ID starting with 'B'. Stored in `bidder` table with status 'Pending'.

2. **How Admin Sees Bids:** Admin navigates to `viewTenderBidsForm.jsp`. This fetches all bids for a specific Tender ID via `BidderDaoImpl.getAllBidsOfaTender()`.
3. **How Winner is Selected:** The Admin clicks the "Accept" button next to a bid. `AcceptBidSrv` runs. It changes the `bidder` status to "Accepted" and inserts a row into `tenderstatus`.

Chapter 14: Complete Request Lifecycle

Let's trace one complete request: **Vendor Updating their Profile**.

1. **Browser (Client):** Vendor changes their Mobile Number in `updateProfile.jsp` and clicks "Update".
2. **Network:** An HTTP POST request is sent to `/UpdateProfileSrv` with parameters (`vname`, `vmob`, etc.).
3. **Servlet (`UpdateProfileSrv.java`):**
   - Retrieves the Session to verify the user is logged in.
   - Reads the form data using `request.getParameter()`.
   - Creates a new `VendorBean` object containing this updated data.
4. **Service / DAO Call:**
   - Servlet calls `VendorDao dao = new VendorDaoImpl();`
   - Calls `dao.updateProfile(vendor);`
5. **DAO (`VendorDaoImpl.java`):**
   - Validates the password first by calling `validatePassword()`.
   - Requests a database connection from `DBUtil`.
   - Creates a `PreparedStatement`: `UPDATE vendor SET vname=?, vmob=?... WHERE vid=?`.
   - Maps the `VendorBean` data to the `?` placeholders.
   - Executes `executeUpdate()`.
6. **Database (MySQL):** Runs the SQL and returns an integer (`1` for success).
7. **DAO Returns:** DAO returns a string message `"Account Updated Successfully!"`.
8. **Servlet Updates Session:** If successful, the Servlet updates the `vendordata` object inside the `HttpSession` so the UI reflects the new mobile number immediately.
9. **View Generation:** The Servlet forwards the request back to `updateProfile.jsp`.
10. **Browser:** User sees an alert: *"Account Updated Successfully!"*

Chapter 15: Problems Faced During Development

(Note: These are genuine architectural observations from the repository that you can mention in an interview).

1. **Date Parsing between Java and MySQL:**
    - *Problem:* Java uses `java.util.Date` but MySQL JDBC requires `java.sql.Date`.
    - *Solution:* I had to meticulously parse Strings from the HTML form using `SimpleDateFormat("yyyy-MM-dd")` into `java.util.Date`, and then convert them into `java.sql.Date` before passing them to the `PreparedStatement` in the DAO.
2. **Direct JDBC Code in Servlets:**
    - *Problem:* In `LoginSrv.java`, JDBC connection code was written directly inside the Servlet to fetch Vendor details quickly. This violated the strict MVC pattern.
    - *Solution:* While it worked for rapid development, I realized the importance of separation of concerns and ensured all other database operations were strictly kept inside the `DAOImpl` classes.
3. **Session Management for Authentication:**
    - *Problem:* Users could bypass the login page by directly typing the URL.
    - *Solution:* I implemented session checking at the very top of every secure JSP page. If the session `user` attribute is null or empty, the page immediately redirects to `loginFailed.jsp`.
4. **Generating Unique IDs:**
    - *Problem:* Relying on MySQL Auto-Increment wasn't ideal for business-readable IDs (like `V2019...` for Vendors).
    - *Solution:* I created a utility class `IDUtil.java` that uses `SimpleDateFormat("yyyyMMddhhmmss")` combined with a prefix to generate globally unique IDs based on timestamps.

Chapter 16: Future Improvements

If an interviewer asks, *"How would you scale this project?"*, here is how you answer:

1. **Spring Boot Migration:** Move away from raw Servlets and JSPs. Spring Boot would eliminate boilerplate code, provide embedded Tomcat, and offer Dependency Injection.
2. **Hibernate (ORM):** Replace raw JDBC `ResultSet` parsing with Hibernate. This maps Java objects directly to database tables, removing the need for manual SQL queries in DAOs.
3. **RESTful API & React/Angular Frontend:** Instead of JSPs rendering HTML on the server, the backend should be a REST API returning JSON. A separate React frontend would consume this API, making the application a Single Page Application (SPA).
4. **JWT Security:** Replace HTTP Sessions with JSON Web Tokens (JWT) for stateless, scalable security, especially if migrating to REST APIs.
5. **Database Connection Pooling:** Instead of creating a new connection via `DriverManager` on every request, use a connection pool like HikariCP to reuse connections and improve performance.

6. **Password Hashing:** Currently, passwords are saved in plain text. I would implement BCrypt hashing before storing passwords in the database.

Chapter 17: Placement Interview Questions (Part 1: Core Java & JDBC)

Here are project-specific questions an interviewer might ask:

1. What is the architecture of your Tender Management System? *Ans:* It follows the MVC (Model-View-Controller) pattern using JSPs for the View, Servlets for the Controller, and Java Beans with DAO classes for the Model.

2. Why did you use interfaces for your DAOs (e.g., `VendorDao`)? *Ans:* To achieve loose coupling. The Servlet only knows about the `VendorDao` interface. If we switch from MySQL to Oracle, we just create a new implementation and the Servlet code remains unchanged.

3. Explain the role of `DBUtil` in your project. *Ans:* It's a utility class that centralizes database connection logic. It reads credentials from a properties file and provides a `static` method to return a `Connection` object.

4. Why do you use `PreparedStatement` instead of `Statement`? *Ans:* `PreparedStatement` prevents SQL Injection by escaping input values. It is also pre-compiled by the database, making it faster when executed multiple times.

5. How are you maintaining sessions in your application? *Ans:* By using the `HttpSession` interface. When a user logs in, I store their role ("admin" or "user") and their data object (`VendorBean`) in the session using `setAttribute()`.

6. What happens if a user directly types the URL `adminHome.jsp` without logging in? *Ans:* At the top of `adminHome.jsp`, a scriptlet checks if the session attribute "user" equals "admin". If not, it calls `response.sendRedirect("loginFailed.jsp")`.

7. How do you generate unique IDs for Vendors and Tenders? *Ans:* I created an `IDUtil` class that formats the current system date and time down to seconds (`yyyyMMddhhmmss`) and prepends a character like 'V' or 'T'.

8. Why does `VendorBean` implement `Serializable`? *Ans:* Because the `VendorBean` object is stored inside the `HttpSession`. Application servers might serialize sessions to disk or send them across a network in a clustered environment, which requires objects to be serializable.

9. In `TenderDaoImpl`, what happens if you forget to close the connection in the `finally` block? *Ans:* It will cause a memory leak. Over time, the database will exhaust its maximum allowed connections, and the application will crash with a "Too many connections" error.

10. How did you handle Date conversion for the database? *Ans:* I parsed the String date from the JSP using `SimpleDateFormat.parse()` into a `java.util.Date`, then used `.getTime()` to convert it into a `java.sql.Date` before inserting it into MySQL.

(Questions 11-30: Servlets and JSP) **11. What is the difference between `sendRedirect` and `RequestDispatcher.forward`?** *Ans:* `forward()` happens on the server side; the URL doesn't change, and the request attributes are preserved. `sendRedirect()` tells the browser to make a completely new request to a new URL.

12. When a vendor updates their profile, how does the UI update immediately? *Ans:* In `UpdateProfileSrv`, after a successful database update, I overwrite the `vendordata` attribute in the `HttpSession` with the newly created `VendorBean` containing the updated details.

13. Can a Servlet handle multiple requests at the same time? *Ans:* Yes. Servlets are multithreaded. The server creates one instance of the Servlet, and each incoming request is handled by a separate thread calling the `doPost` or `doGet` method.

14. Are your DAOs thread-safe? *Ans:* Yes, because the database `Connection`, `PreparedStatement`, and `ResultSet` objects are created locally inside the method, meaning each thread gets its own copy.

15. What is the purpose of `web.xml` in your project? *Ans:* It is the deployment descriptor. In this project, it is primarily used to configure `index.jsp` as the welcome file list. Servlets are mapped using the `@WebServlet` annotation instead of `web.xml`.

16. Why did you use `@WebServlet("/LoginSrv")`? *Ans:* This is an annotation introduced in Servlet 3.0. It maps the URL `/LoginSrv` directly to the `LoginSrv` class, eliminating the need to write XML mappings in `web.xml`.

17. How does the frontend communicate with the backend? *Ans:* Using HTML forms with the `method="post"` attribute. The `action` attribute is set to the mapped URL of the Servlet.

18. Why did you use JSPs instead of standard HTML? *Ans:* JSPs allow embedding Java code (scriptlets) and expressions to display dynamic data, like showing the specific Vendor's name from the session.

19. What is a scriptlet in JSP? *Ans:* It's a block of Java code enclosed in `<% ... %>`. I used scriptlets at the top of my JSPs to check session validation.

20. What is `response.setContentType("text/html")`? *Ans:* It tells the client's browser that the server is sending back an HTML document, so the browser knows how to parse and render it.

(Questions 21-40: Database & SQL) **21. Explain your database schema.** *Ans:* It consists of 5 tables: `vendor` (user details), `tender` (project details), `bidder` (bids placed), `tenderstatus` (linking winning bids to tenders), and `notice` (announcements).

22. What is the relationship between the `tender` and `bidder` tables? *Ans:* One-to-Many. One Tender can have many Bids, but each Bid belongs to exactly one Tender. The `bidder` table has a foreign key `tid` referencing the `tender` table.

23. How do you prevent a Tender from being assigned to multiple vendors? *Ans:* In `TenderDaoImpl.assignTender()`, before inserting into `tenderstatus`, I first execute a `SELECT` query to check if that `tid` already exists. If it does, I return a failure message.

24. Why did you use `executeQuery()` vs `executeUpdate()`? *Ans:* `executeQuery()` is used for `SELECT` statements because it returns a `ResultSet`. `executeUpdate()` is used for `INSERT`, `UPDATE`, and `DELETE` because it returns an integer representing the number of rows affected.

25. How do you handle database credentials securely? *Ans:* Instead of hardcoding them in Java, I placed them in a properties file (`dbdetails.properties`). `DBUtil` reads this file using `ResourceBundle`.

26. What happens if the database server goes down? *Ans:* `DriverManager.getConnection()` will throw a `SQLException`. In the DAO, this exception is caught in the `catch` block, printed to the stack trace, and a failure message is returned to the UI.

27. What is a `ResultSet`? *Ans:* It is an object that maintains a cursor pointing to a row of a table generated by executing a database query. We use `rs.next()` to iterate through the rows.

28. Why do you use `while(rs.next())` in some methods and `if(rs.next())` in others? *Ans:* `while(rs.next())` is used when fetching multiple rows. `if(rs.next())` is used when we only expect a single row.

29. How is the 'Notice' table's Primary Key different from 'Vendor'? *Ans:* The `notice` table uses an `AUTO_INCREMENT` integer for its ID, generated by MySQL. The `vendor` table uses a custom String ID generated by Java (`IDUtil`).

30. Did you use any connection pooling? *Ans:* No, this project creates a direct connection for every request using `DriverManager`. For an enterprise application, I would upgrade this to use a connection pool like HikariCP.

(Questions 31-50: Business Logic) **31. Can a Vendor bid on a Tender that has already been assigned?** *Ans:* In the current schema, a vendor could technically submit a bid. An improvement would be to check the `tenderstatus` table before allowing a new bid to be placed.

32. What is the difference between `Admin` and `Vendor` roles? *Ans:* Admins create tenders, post notices, and accept bids. Vendors register, view tenders, submit bids, and manage their profiles.

33. How does the system know who won the bid? *Ans:* When the admin clicks "Accept", `AcceptBidSrv` updates the bid status in the `bidder` table to "Accepted" and inserts a record linking the `tid` and `vid` into the `tenderstatus` table.

34. Why do you have a `TenderStatusBean`? *Ans:* It represents a row in the `tenderstatus` table, making it easy to fetch a list of all assigned tenders and display them on the JSP page.

35. If a vendor updates their password, what happens to their active session? *Ans:* In `ChangePasswordSrv`, if the update is successful, the Java code calls `vendor.setPassword(newPass)` and updates the session object, so the session remains valid but holds the new password.

36. Is the password stored securely? *Ans:* Currently, it is stored as plain text. As a future enhancement, I would hash the password using BCrypt before storing it in the database.

37. What happens if two vendors register with the same email? *Ans:* `VendorDaoImpl.registerVendor()` checks if the email exists using a `SELECT` query before inserting. If it exists, it returns a message: "Registration Declined! Email Id already Registered".

38. Why are you trimming input parameters (e.g., `request.getParameter("username").trim()`)? *Ans:* To remove any accidental leading or trailing white spaces entered by the user, which could cause login failures or database mismatches.

39. How do you implement "Logout"? *Ans:* `LogoutSrv` overwrites the session attributes `user`, `username`, and `password` to empty strings, effectively killing the user's access rights.

40. What is the use of `RequestDispatcher.include()` in your project? *Ans:* Instead of redirecting to a new page, `include()` embeds the response of the target JSP into the current response. I use this to show success/failure messages on the same form page using JavaScript injections.

41. Why did you use `PrintWriter.print("<script>...</script>")`? *Ans:* To dynamically inject a JavaScript command from the Servlet to update an HTML element on the JSP page after a form submission.

42. How are you closing resources if an exception occurs? *Ans:* The `DBUtil.closeConnection()` methods are always placed inside a `finally` block in the DAOs, ensuring they run regardless of whether an exception occurred or not.

43. Is `DBUtil.closeConnection()` safe to call on a null object? *Ans:* Yes, the method explicitly checks `if (con != null)` before calling `con.close()`, preventing `NullPointerExceptions`.

44. What happens if a Vendor types a letter into the "Price" field? *Ans:* The HTML form likely handles it if `type="number"`. If it bypasses the frontend, `Integer.parseInt(stprice)` in the Servlet will throw a `NumberFormatException`.

45. How do you display a list of Tenders on the JSP? *Ans:* The JSP calls `TenderDao.getAllTenders()`, which returns a `List<TenderBean>`. The JSP then uses a `for` loop to

iterate through the list and generate HTML table rows.

46. Can a Vendor bid multiple times on the same Tender? *Ans:* The system currently generates a new unique Bid ID for every submission, so technically yes. To prevent this, I would add a database check constraint or a query in `bidTender()` to verify if the vendor has already bid.

47. Why are you separating `BidderBean` and `TenderStatusBean`? *Ans:* `BidderBean` represents the application (Bid) submitted by a vendor. `TenderStatus` represents the final decision (Assignment) of the project. A tender has many bids, but only one final status.

48. Why is the `user` attribute set to "user" for vendors? *Ans:* It's used as a generic role identifier in the session to distinguish between the "admin" role and standard vendor roles for access control on JSP pages.

49. How do you handle exceptions in JDBC? *Ans:* Using try-catch blocks. If a `SQLException` occurs, I print the stack trace for debugging and append `e.getMessage()` to the status string returned to the user interface.

50. If you were to redo this project, what is the FIRST thing you would change? *Ans:* I would move all the hardcoded SQL queries from Java Strings into an ORM like Hibernate, and implement proper password hashing for security.

Chapter 17: Placement Interview Questions (Part 2: Deep Dives & General Concepts)

(Questions 51-100)

51. How does the system recognize an Admin vs a Vendor? *Ans:* In `LoginSrv`, the HTML form sends a parameter named `user` depending on which submit button was clicked ("Login as Admin" vs "Login as Vendor"). The backend reads this and routes the authentication logic accordingly.

52. Is hardcoding "Admin" credentials in the Servlet a good practice? *Ans:* No. It was done for simplicity here, but in production, Admin credentials should be stored securely in the database with role-based access control (RBAC).

53. Explain the `<marquee>` tag used in your JSPs. *Ans:* It's an HTML tag used for scrolling text (like the "Welcome" banner). However, it is obsolete in HTML5 and should ideally be replaced with CSS animations.

54. What happens if a vendor inputs an SQL query into the login field? *Ans:* Because I used `PreparedStatement` and set parameters via `setString()`, the input is escaped. The SQL query will be treated as a literal string, preventing SQL Injection.

55. How do you pass data from a Servlet to a JSP? *Ans:* I used `HttpSession.setAttribute()`. Alternatively, I could use `HttpServletRequest.setAttribute()` combined with `RequestDispatcher.forward()`.

56. What does `ResultSet.next()` actually do? *Ans:* It moves the cursor forward one row from its current position. It returns `true` if a new row is valid, and `false` if there are no more rows.

57. How would you handle a situation where two admins try to accept different bids for the same tender simultaneously? *Ans:* I would implement Database Transactions (`con.setAutoCommit(false)`) and Row Locking (e.g., `SELECT ... FOR UPDATE`) to ensure isolation. Currently, the first one to execute `INSERT INTO tenderstatus` wins, but the second might cause a duplicate entry or invalid state.

58. Why do you use `.trim()` on all request parameters? *Ans:* To remove white spaces at the beginning or end of user inputs, ensuring that accidental spaces don't cause authentication failures.

59. What design pattern does your DAO layer follow? *Ans:* It follows the Data Access Object (DAO) pattern, which is a structural pattern to isolate the application/business layer from the persistence layer using an abstract API (the Interface).

60. Why did you use `java.util.List` to return multiple records? *Ans:* `List` is an interface that allows us to return an ordered collection of objects (like `ArrayList`). It's flexible and allows the UI to easily iterate over the elements.

61. What is the scope of variables declared inside the `doPost` method? *Ans:* They are local to that specific request thread. They are destroyed once the `doPost` method finishes executing.

62. If you declare a variable outside `doPost` in a Servlet, what happens? *Ans:* It becomes an instance variable. Since Servlets are singletons (one instance shared among all threads), instance variables are NOT thread-safe and can lead to data corruption if multiple users access it simultaneously.

63. How do you delete a Notice from the system? *Ans:* Admin clicks remove, `RemoveNoticeSrv` reads the `noticeid`, and passes it to `NoticeDaoImpl.removeNotice()`, which executes a `DELETE FROM notice WHERE id=?`.

64. Can a deleted record be recovered in your current database? *Ans:* No. A `DELETE` statement permanently removes the row. A better approach for enterprise apps is "Soft Delete"—adding an `isActive` boolean column and setting it to false instead of deleting the row.

65. What is the difference between `java.util.Date` and `java.sql.Date`? *Ans:* `util.Date` contains both date and time information. `sql.Date` is a wrapper around a millisecond value used by JDBC to identify a SQL DATE type, which only contains year, month, and day.

66. Why didn't you use Spring for this project? *Ans:* This project was built to understand the core foundations of Java EE (Servlets, JSP, JDBC). Using frameworks like Spring hides these internal workings.

67. What is Maven and why did you use a `pom.xml`? *Ans:* Maven is a build automation and dependency management tool. The `pom.xml` file dictates how the project is built and automatically downloads dependencies like `mysql-connector-java`.

68. How do you handle 404 or 500 errors? *Ans:* I included `errorPage="errorpage.jsp"` in the JSP directives. If an uncaught exception occurs during JSP rendering, it automatically forwards to the error page.

69. What is the role of the Tomcat server? *Ans:* Apache Tomcat is a Web Container. It listens for HTTP requests, creates the `HttpServletRequest` and `HttpServletResponse` objects, routes the request to the correct Servlet, and sends the response back.

70. Explain the `serialVersionUID` in your Beans. *Ans:* It is a unique identifier for Serializable classes. It ensures that during deserialization, the loaded class exactly matches the serialized object.

71. How do you display images in your JSPs? *Ans:* By placing them in the `WebContent/images` folder and referencing them using standard HTML `<img>` tags relative to the web context.

72. What CSS framework did you use? *Ans:* I used Bootstrap to make the UI responsive and well-structured quickly.

73. What is the purpose of `RequestDispatcher.include()`? *Ans:* It includes the content of a specified resource (like another JSP) in the response. I used it to reload a form page while injecting a success/error message below it via JavaScript.

74. How does the "Welcome back, [Name]" text work? *Ans:* The `VendorBean` object is pulled from the session using `<%= ((VendorBean)session.getAttribute("vendordata")).getName() %>`.

75. What happens if the database connection string is wrong? *Ans:* `DriverManager.getConnection()` will fail, throwing a `SQLException`. The application won't crash entirely due to try-catch blocks, but database operations will fail and return error strings.

76. How would you implement pagination for Tenders if there were 10,000 of them? *Ans:* I would modify the SQL query to use `LIMIT` and `OFFSET`. The Servlet would accept a `pageNumber` parameter and calculate the correct offset to fetch only 10 or 20 rows at a time.

77. Why did you use `java.text.SimpleDateFormat`? *Ans:* To convert the string date coming from the HTML `<input type="date">` (format `yyyy-MM-dd`) into a Java `Date` object that the backend can manipulate.

78. Is `SimpleDateFormat` thread-safe? *Ans:* No. Therefore, I instantiate a new `SimpleDateFormat` object locally inside methods rather than declaring it globally.

79. How do you handle a scenario where a user submits a form twice rapidly? *Ans:* In this basic project, it might create duplicate records (if no unique constraint blocks it). To prevent this, I could implement the PRG (Post/Redirect/Get) pattern or use unique tokens.

80. What are Foreign Keys and where did you use them? *Ans:* They enforce referential integrity. In the `bidder` table, `vid` references the `vendor` table, meaning a bid cannot exist for a non-existent vendor.

81. Can you explain the `executeUpdate` return value? *Ans:* It returns an `int` representing the number of rows affected by the query. If it returns 0, no rows were updated/inserted.

82. Why is your HTML outputting `<script>` tags from the Servlet? *Ans:* It was a quick way to pass a success/failure message back to the DOM without reloading the entire page or using AJAX.

83. How would you replace the `<script>` injection with a better approach? *Ans:* I would set an attribute in the request (e.g., `request.setAttribute("message", status)`) and use JSP Expression Language (`${message}`) to display it cleanly in the HTML.

84. What is the role of `Class.forName()` in DBUtil? *Ans:* It dynamically loads the JDBC driver class into memory, registering it with the `DriverManager` so connections can be made.

85. How do you prevent XSS (Cross-Site Scripting) in this app? *Ans:* Currently, it is vulnerable if a user inputs JavaScript into a text field. To fix this, I would use a library like OWASP Java Encoder to sanitize outputs before rendering them in JSPs.

86. What is the difference between `doGet` and `doPost`? *Ans:* `doGet` handles GET requests (data in URL, used for fetching). `doPost` handles POST requests (data in body, used for submitting sensitive data like passwords). In my Servlets, I map `doGet` to call `doPost`.

87. Why do you map `doGet` to `doPost`? *Ans:* To allow the Servlet to process the same logic regardless of how the form is submitted, centralizing the code.

88. Where are the database queries executed? On the Web Server or Database Server? *Ans:* The query string is constructed on the Web Server (Tomcat) and sent over the network to the Database Server (MySQL), which parses and executes it, returning the result.

89. How do you handle Null inputs from the frontend? *Ans:* I use `.trim()` which would throw a `NullPointerException` if the parameter is entirely missing. Adding a null check (e.g., `if (str !=`

`null)`) before trimming would make it more robust.

90. What is a "Stateless" protocol? *Ans:* HTTP is stateless; every request is independent. We use Cookies and Sessions to simulate state and remember logged-in users.

91. How does Tomcat know which Session belongs to which user? *Ans:* When a session is created, Tomcat sends a Cookie named `JSESSIONID` to the browser. The browser sends this cookie back on every subsequent request, allowing Tomcat to match it.

92. What is the purpose of `vendor.getPassword()` if you never display it? *Ans:* It is used internally to validate old passwords when the vendor attempts to change their password via `ChangePasswordSrv`.

93. Why did you use `toLowerCase()` on emails during registration? *Ans:* To ensure emails are case-insensitive. `User@email.com` and `user@email.com` will be treated as the exact same identity in the database.

94. What is the `id` column in the `notice` table? *Ans:* It is an auto-incrementing integer. I let the database handle ID generation for notices because they don't require business-readable IDs like vendors do.

95. How does the `getAllBidsOfaTender` method work? *Ans:* It executes `SELECT * FROM bidder WHERE tid=?`, loops through the `ResultSet`, creates a `BidderBean` for each row, adds them to an `ArrayList`, and returns the list to the UI for display.

96. What is the difference between a Web Server and an Application Server? *Ans:* A web server handles HTTP requests and serves static files. An application server (like Tomcat, though it's technically a Web Container) can execute server-side code like Servlets and JSPs.

97. How do you deploy this application? *Ans:* By compiling it into a `.war` (Web Archive) file using Maven and placing it in Tomcat's `webapps` folder.

98. Could this application run on PostgreSQL instead of MySQL? *Ans:* Yes. You would just need to change the Driver Class Name, Connection URL, and Username/Password in `dbdetails.properties`, and include the Postgres JAR.

99. What was the hardest bug to fix in this project? *Ans:* (Personalize this answer!) Usually, it involves dealing with `NullPointerExceptions` when session data expires, or SQL errors related to Date parsing and Foreign Key constraints failing during Bid acceptance.

100. Are you proud of this project? *Ans:* Yes. It allowed me to understand the raw mechanics of web development. While modern frameworks hide this complexity, building it with raw Servlets and JDBC taught me how things actually work under the hood.

Chapter 18: 2 Minute Project Explanation

(Perfect for starting an interview)

"My project is a **Tender Management System**, a web application built using **Core Java, Servlets, JSP, and MySQL**. It digitizes the traditional paper-based government procurement process.

The architecture strictly follows the **MVC design pattern**.

- The **View** is built with HTML, CSS, Bootstrap, and JSP.
- The **Controller** logic is handled by Java Servlets.
- The **Model** consists of Java Beans and DAO (Data Access Object) classes, keeping the business logic decoupled from the database operations.

The system handles two main roles: **Admins** and **Vendors**. Admins can float tenders, post public notices, and review incoming bids. Vendors can register, browse available tenders, and submit bids electronically. I implemented custom session management to secure the routes, preventing unauthorized URL access. To ensure security at the database level, I used **PreparedStatements** extensively to block SQL injection attacks.

Overall, it is a robust, scalable foundation that demonstrates my strong grasp of Object-Oriented Programming, MVC architecture, and relational database management."

Chapter 19: 5 Minute Project Explanation

(For a deep dive when the interviewer asks for details)

Introduction: "I developed a Tender Management System to solve the inefficiencies of manual government procurement. It is an end-to-end web application utilizing Java EE technologies (Servlets/JSP) on the backend and MySQL for persistence.

Architecture (MVC): I structured the project using the MVC pattern.

- For the **Model**, I separated data carriers (Java Beans like `VendorBean`, `TenderBean`) from the database logic (DAO classes).
- I used a `DBUtil` class to manage connections dynamically via a `.properties` file, ensuring credentials aren't hardcoded in Java.
- The **Controllers** are Servlets (`LoginSrv`, `CreateTenderSrv`, etc.) that intercept HTTP requests, process data, invoke the DAOs, and forward the request.
- The **View** consists of JSPs that use session attributes to dynamically display content.

Key Features & Workflows:

1. **Authentication & Authorization:** I built a custom login system for Admins and Vendors. I utilized `HttpSession` to maintain state. To secure the app, every private JSP page checks the session before rendering, redirecting unauthorized users.
2. **The Bidding Engine:** Admins create tenders which are stored in the database. Vendors can browse these and submit bids (including price and deadline). This requires complex relationships in the database—specifically utilizing Foreign Keys between the `vendor`, `tender`, and `bidder` tables.
3. **Tender Assignment:** The Admin can view a list of all bids for a specific tender. Upon clicking 'Accept', the DAO updates the bid status, and a transaction-like process assigns the tender in the `tenderstatus` table.

Technical Challenges Overcome: A major challenge was handling data types between Java and SQL, specifically Dates. Forms submit Strings, Java utilizes `java.util.Date`, and MySQL requires `java.sql.Date`. I implemented a robust parsing mechanism in my Beans to handle this conversion seamlessly. I also focused heavily on security by exclusively using `PreparedStatement` interfaces to prevent SQL injection.

Future Scope: While this project solidified my understanding of core web mechanics, if I were to scale it for enterprise use, I would migrate it to Spring Boot for better dependency management, use Hibernate ORM to replace raw JDBC, and switch from HTTP Sessions to stateless JWT authentication."

Chapter 20: Final Revision Notes (Cheat Sheet)

Concept	How it's used in this Project
MVC	JSP (View), Servlets (Controller), DAO/Beans (Model).
DAO Pattern	`VendorDao` (Interface) → `VendorDaoImpl` (Class). Separates SQL from Java.
Session	`HttpSession session = request.getSession();` Used to remember who logged in.
PreparedStatement	Used in every DAO method to stop SQL Injection. (`?` placeholders).
Properties File	`dbdetails.properties`. Stores DB user/pass. Read by `ResourceBundle`.

Concept	How it's used in this Project
Scriptlets	`<% ... %>` inside JSPs to run Java code (e.g., Session validation).
web.xml	Configures `index.jsp` as the starting page.
JDBC Steps	1. Load Driver (`Class.forName`) 2. Get Connection 3. Prepare Statement 4. Execute (`executeQuery` / `executeUpdate`) 5. Close Connection.
Foreign Keys	`bidder` table uses `vid` and `tid` to ensure bids are linked to valid vendors and tenders.

Best of luck with your interviews! You know this project inside and out.